

Trabajo Final — KNN vs LSTM

1. Introducción

Este trabajo final se basa en el artículo *Deep Learning vs. Machine Learning for Intrusion Detection in Computer Networks: A Comparative Study* (Ali et al., 2025, *Applied Sciences*, 15(4), 1903), en el cual se comparan modelos de aprendizaje automático clásicos contra modelos de aprendizaje profundo para la detección de intrusiones en redes de computadoras, utilizando el dataset CICIDS-2017.

El objetivo es que el estudiante replique de forma crítica y documentada la comparación entre **KNN (K-Nearest Neighbors)** (modelo clásico) y **LSTM (Long Short-Term Memory)** (modelo de aprendizaje profundo), analice los resultados obtenidos y los contraste con los reportados en el paper.

2. Dataset

El dataset utilizado es el **CICIDS-2017** (Canadian Institute for Cybersecurity Intrusion Detection Evaluation Dataset 2017), disponible en:

<https://www.unb.ca/cic/datasets/ids-2017.html>

El dataset contiene tráfico de red capturado durante cinco días, incluyendo tráfico normal y distintos tipos de ataques. En su forma original cuenta con 2.830.743 registros y 79 columnas, incluyendo una columna de etiqueta (*Label*) con las categorías de tráfico.

3. Preprocesamiento

Seguir los pasos de preprocesamiento descritos en el paper. Se indica el pseudocódigo y los valores de referencia obtenidos por los autores.

Paso 1 — Eliminar duplicados

Identificar y eliminar registros duplicados. El paper reporta **308.381 duplicados** eliminados.

Paso 2 — Tratar valores faltantes

Identificar columnas con valores nulos e imputar con la media de cada columna. El paper reporta **353 valores faltantes**.

Paso 3 — Eliminar valores infinitos

Algunas columnas contienen valores $\pm\infty$ producto del cálculo de *features* de red. Eliminar las filas que los contengan.

Paso 4 — Consolidar etiquetas

La columna *Label* contiene variantes textuales de los tipos de ataque con diferencias en mayúsculas, espacios y guiones según el archivo CSV de origen (por ejemplo: “Web Attack – Brute Force”, “Web Attack – XSS”, “Web Attack – Sql Injection”). Para simplificar la clasificación, consolidar las etiquetas similares bajo una categoría común. Se provee el mapeo a utilizar:

- etiquetas que contienen “Web Attack” → “Web Attack”
- etiquetas que contienen “DoS” → “DoS”
- etiquetas que contienen “DDoS” → “DDoS”
- etiquetas que contienen “PortScan” → “PortScan”
- etiquetas que contienen “Bot” → “Bot”
- “BENIGN” → “BENIGN”
- resto → “Other”

Nota: usar `str.contains()` con expresiones regulares simples o `str.strip()` para normalizar espacios antes de aplicar el mapeo.

Paso 5 — Escalado de *features*

Aplicar estandarización (media 0, desvío 1) a todas las columnas numéricas excepto la etiqueta. Ajustar el `StandardScaler` sobre el conjunto de entrenamiento y aplicar la transformación a *train* y *test*.

Paso 6 — Balanceo de clases con SMOTE

El dataset es altamente desbalanceado (la clase BENIGN domina ampliamente). Aplicar SMOTE (Synthetic Minority Over-sampling Technique) **solo sobre el conjunto de entrenamiento**, luego de separar en *train/test* con estratificación por etiqueta.

Paso 7 — Selección de *features* por correlación

Calcular la matriz de correlación entre *features* y eliminar aquellas con correlación absoluta superior a 0,85 con otra *feature*, conservando solo una de cada par altamente correlacionado.

4. Marco teórico

Elaborar una explicación teórica de cada uno de los dos algoritmos asignados: **KNN (K-Nearest Neighbors)** y **LSTM (Long Short-Term Memory)**. La explicación debe incluir:

- Intuición del algoritmo: ¿qué problema resuelve y cómo lo aborda?
- Arquitectura o estructura del modelo (con especial énfasis en la arquitectura para los modelos de aprendizaje profundo).
- Hiperparámetros principales y su efecto sobre el modelo.
- Fortalezas y limitaciones conocidas del algoritmo.

Bibliografía obligatoria:

- Para **KNN (K-Nearest Neighbors)**: Hastie, T., Tibshirani, R. & Friedman, J. — *The Elements of Statistical Learning* (<https://hastie.su.domains/ElemStatLearn/>). Ver Capítulo 13, Sección 13.3 (K-Nearest Neighbors).
- Para **LSTM (Long Short-Term Memory)**: Goodfellow, I., Bengio, Y. & Courville, A. — *Deep Learning* (<https://www.deeplearningbook.org/>). Ver Capítulo 10, Sección 10.10 (LSTM).
- Ali, M.L. et al. (2025). Deep Learning vs. Machine Learning for Intrusion Detection in Computer Networks: A Comparative Study. *Applied Sciences*, 15(4), 1903.

5. Implementación

Implementar ambos modelos (**KNN (K-Nearest Neighbors)** y **LSTM (Long Short-Term Memory)**) en un Jupyter Notebook, siguiendo el pipeline de preprocesamiento descrito en la sección 3. El notebook debe ser **reproducible** y **auto-contenido**: cualquier persona debe poder ejecutarlo de principio a fin sin intervención manual, y las celdas de texto (markdown) deben explicar cada etapa.

Para cada modelo, reportar: *accuracy*, F1-score (macro y ponderado), *precision* y *recall*.

6. Evaluación y comparación con el paper

Contrastar los resultados obtenidos con los reportados en Ali et al. (2025). Los valores de referencia del paper son:

Métrica	KNN (K-Nearest Neighbors)	LSTM (Long Short-Term Memory)
Accuracy	99,36 %	98,00 %
F1-score (macro)	97,62 %	84,00 %
F1-score (ponderado)	99,00 %	98,00 %

Discutir brevemente las diferencias entre los resultados propios y los del paper, identificando posibles causas (versión del dataset, hiperparámetros, semilla aleatoria, etc.).

7. Análisis crítico

Una de las conclusiones más llamativas del paper es que los modelos clásicos de aprendizaje automático (en particular Random Forest y Decision Tree) superan a los modelos de aprendizaje profundo en este dataset, a pesar de que el aprendizaje profundo suele asociarse con resultados de vanguardia en muchas tareas.

Elaborar un análisis crítico de este fenómeno **para el par asignado (KNN vs LSTM)**, respondiendo al menos las siguientes preguntas:

1. ¿Qué resultado obtiene cada modelo (KNN (K-Nearest Neighbors) vs LSTM (Long Short-Term Memory)) en términos de *accuracy* y F1? ¿Cuál supera al otro en este experimento?
2. ¿Por qué crees que el modelo clásico obtiene ese resultado en un dataset tabular estructurado como CICIDS-2017? ¿Qué características del dataset favorecen o perjudican a cada tipo de modelo?

3. ¿En qué tipo de problemas o datos esperarías que el modelo de aprendizaje profundo superara al clásico? Justificar.
4. ¿Qué rol juega el costo computacional en la elección de un modelo para un sistema de detección de intrusiones en producción?

8. ¿Cuándo gana el aprendizaje profundo?

El resultado del paper no implica que LSTM (Long Short-Term Memory) sea un modelo inferior en general. Los modelos de aprendizaje profundo tienen ventajas estructurales en ciertos tipos de datos y problemas. El objetivo de esta sección es identificar y demostrar un caso donde **LSTM** (Long Short-Term Memory) supere claramente a **KNN** (K-Nearest Neighbors).

Opciones (elegir una):

- **Opción A — Paper real:** buscar en la literatura un paper donde el modelo de aprendizaje profundo asignado supere claramente al modelo clásico asignado. Citar el paper, describir el dataset utilizado y los resultados reportados. Justificar por qué ese tipo de dato favorece al aprendizaje profundo.
- **Opción B — Toy example:** construir un dataset sintético donde el modelo de aprendizaje profundo tenga una ventaja estructural demostrable (por ejemplo, datos con dependencias temporales, patrones espaciales, o relaciones no lineales complejas). Implementar ambos modelos sobre ese dataset en el notebook y mostrar empíricamente que el aprendizaje profundo gana.

En ambos casos, justificar:

- ¿Qué características del dato o del problema favorecen al modelo de aprendizaje profundo?
- ¿Por qué el modelo clásico tiene dificultades en ese contexto?
- ¿Qué conclusión general se puede extraer sobre cuándo conviene usar cada tipo de modelo?

Esta sección debe estar presente tanto en el notebook (con código y análisis) como en las slides Beamer (con resultados y conclusiones).

9. Presentación Beamer

Preparar una presentación en LaTeX/Beamer que acompañe el notebook. La presentación debe tener entre **20 y 30 slides** y seguir la siguiente estructura:

1. **Portada.** Título del trabajo, nombre del estudiante, par de algoritmos asignado, curso y fecha.
2. **Introducción al problema.** Motivación de la detección de intrusiones en redes. Presentación del paper de referencia y su pregunta central. ¿Por qué comparar ML clásico vs aprendizaje profundo?
3. **Descripción del dataset.** Qué es CICIDS-2017, cómo fue generado, qué tipos de ataques contiene. Distribución de clases antes y después de SMOTE.
4. **Explicación teórica de los algoritmos.** Una sección por algoritmo (KNN (K-Nearest Neighbors) y LSTM (Long Short-Term Memory)). Incluir intuición, arquitectura (con espe-

cial detalle para el modelo de aprendizaje profundo) e hiperparámetros principales. Apoyarse en figuras o diagramas cuando sea posible.

5. **Pipeline de preprocesamiento.** Resumen visual de los pasos aplicados: eliminación de duplicados, imputación, valores infinitos, consolidación de etiquetas, escalado, SMOTE y selección de *features*.
6. **Resultados obtenidos vs. paper.** Tabla comparativa de métricas propias vs. las del paper. Gráficos de matriz de confusión para cada modelo.
7. **Análisis crítico.** Discusión sobre por qué los clásicos superan al aprendizaje profundo en CICIDS-2017. Reflexión sobre las condiciones bajo las cuales el aprendizaje profundo sería preferible.
8. **¿Cuándo gana el aprendizaje profundo?** Presentación del paper encontrado o del toy example construido. Resultados comparativos y conclusión general sobre cuándo usar cada tipo de modelo.

La presentación debe entregarse como archivo .tex y .pdf. Debe ser auto-contenida: alguien que no haya visto el notebook debe poder comprender el trabajo leyendo solo las slides.

10. Estructura del repositorio (entrega en GitHub)

El trabajo se entrega como un repositorio de GitHub, siguiendo la estructura de proyecto vista en la Unidad 6 (Bloque 5: Portfolio profesional):

```
mi_proyecto/
  README.md
  data/
    raw/                (datos originales, nunca modificados)
    processed/          (datos limpios, listos para modelar)
  notebooks/            (numerados por orden de ejecución)
  src/
    utils.py             (funciones reutilizables)
  reports/               (la presentación Beamer compilada)
  requirements.txt
```

El README.md debe seguir las siete secciones mínimas vistas en clase: (1) título y descripción breve; (2) motivación; (3) datos (origen, tamaño, variables principales); (4) metodología (qué técnicas se usaron y por qué); (5) resultados principales; (6) cómo ejecutar; (7) limitaciones y próximos pasos.

Si los datos son grandes o sensibles, subir solo una muestra o las instrucciones para obtenerlos.

11. Entregables

- `tpfinal_mad1_knn_lstm.ipynb` — Jupyter Notebook completo y reproducible
- `tpfinal_mad1_knn_lstm.tex` — Código fuente de la presentación Beamer
- `tpfinal_mad1_knn_lstm.pdf` — PDF compilado de la presentación
- `tpfinal_mad1_knn_lstm_chat.json` — Conversación completa con la herramienta de IA utilizada. (NO SE SUBE AL REPOSITORIO, SINO QUE SE ENVIA POR MAIL)

12. Documentación del uso de IA

El uso de herramientas de IA (ChatGPT, Claude, etc.) está permitido durante el desarrollo del trabajo.

- Utilizar una **única sesión de chat** para todo el desarrollo del trabajo, de principio a fin.
- Al finalizar, exportar esa conversación completa y entregarla junto con el resto de los archivos. En ChatGPT: Configuración → Exportar datos, se entrega el archivo `conversations.json`. Alternativamente, el chat en texto plano.
- Nombrar el archivo como `tpfinal_mad1_knn_lstm_chat.json` (o `.txt`).

13. Criterios de evaluación

La mitad de la nota depende del **Jupyter Notebook**, evaluado en base a:

- Reproducibilidad: el notebook corre de principio a fin sin intervención.
- Auto-contenido: las celdas markdown explican cada etapa sin necesidad de consultar el paper.
- Claridad del código: organizado, comentado y sin celdas innecesarias.
- Correctitud de resultados: métricas comparables a las del paper.

La otra mitad depende de la **presentación Beamer**, evaluada en base a:

- Claridad: cada slide transmite una sola idea principal.
- Explicabilidad: los algoritmos quedan explicados para alguien que no los conoce.
- Auto-contenida: la presentación se entiende sola, sin el notebook.
- Calidad de la presentación: uso correcto del template, sin exceso de texto.

La organización del repositorio (estructura de carpetas y README) también se considera en la evaluación, en línea con lo visto en la Unidad 6.

A criterio del docente, se podrá solicitar una presentación oral como instancia de desempate o verificación.